

NER on Spark: Data Paths and Execution Internals

Where every byte comes from and goes to, and exactly how Spark runs the NER pipeline - driver, scheduler, executor, Python worker, model server and GPU - traced on the air-gapped simulation in service mode and in cluster mode.

Traced run: `ner_trace_20260927_142727` (`docs/ner_internals/run_trace.sh`). Inputs: the 6 documents in `data/ner_samples`. Models: warm node caches of the HDFS model store. Evidence used: Spark event logs, executor logs, driver logs, model-server log, a `/proc` probe in every container (processes, sockets, network counters, 1-2 samples/s), `nvidia-smi` samples (2/s) and a stage profile of the NER code on the GPU. Machine: laptop, Intel i5-9300H, 6 GB for Docker/WSL2, NVIDIA GTX 1650 4 GB, PCIe Gen3 x16.

Contents

1. The short answer - ten data paths
2. Where the data comes from
3. Driver side: from the command line to a Spark job
4. Worker side: executor JVM and Python worker
5. Inside the model process: NER step by step on the GPU
6. Service mode end to end - measured timeline
7. Cluster mode end to end - measured timeline
8. Service mode vs cluster mode
9. Nitty-gritty findings
10. Reproduce

1. The short answer - ten data paths

Three kinds of data move: **models** (large, move once per node), **document names** (tiny, move through Spark) and **document contents and results** (read and produced where the model runs). Spark itself never carries a document: it carries file names to the process that holds the models, and carries the result dictionaries back.

25. NER on Spark - every data movement path

(n) = path number in the text. Service mode: the kitchen reads the documents; cluster mode: the Python worker does (the primed paths)

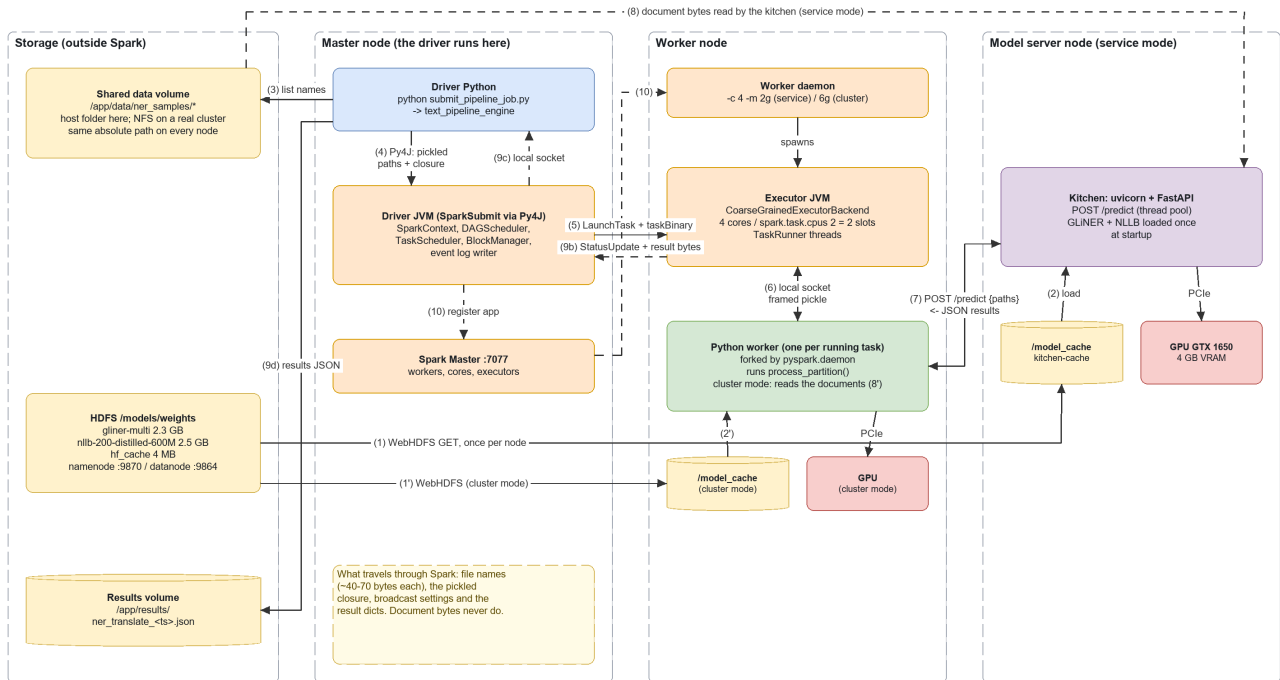


Figure 1 - every data path; the numbers match the table below

#	what moves	from -> to	transport and format	size (measured)	when
1	model folders	HDFS datanode -> node cache (/model_cache)	WebHDFS HTTP: namenode 307 redirect, datanode streams 8 MB chunks; .partial dir -> rename	4.8 GB (gliner 2.3 GB, nllb 2.5 GB)	first load on a node only
2	model weights	node cache -> CPU RAM -> GPU VRAM	from_pretrained reads the files; .to('cuda') copies every tensor over PCIe	GLiNER 1.2 GB fp32 -> 577.9 MB fp16; NLLB 2.5 GB	every model load: kitchen start; every task in cluster mode
3	file names	shared data volume -> driver	directory listing (glob), then os.path.abspath	6 names	each submit
4	names + code	driver Python -> driver JVM	Py4J commands over localhost TCP; path batches pickled to a temp file; closure pickled (cloudpickle)	2 batches of 3 paths; closure in the task binary	each job
5	tasks	driver JVM -> executor JVM	Netty RPC LaunchTask; task binary and settings as TorrentBroadcast blocks fetched from the driver's BlockManager	task binary 5.2 KiB + 3 settings broadcasts of 238.0 B, 215.0 B, 201.0 B	each task
6	names in, results out	executor JVM <-> Python worker	localhost TCP to pyspark.daemon (forked worker); length-prefixed pickle frames	in: header + closure + 3 paths; out: one result dict	each task
7	HTTP request / response	Python worker <-> kitchen (service mode)	HTTP/1.1 POST /predict, JSON	request 134 B body, response 18.8 KB	each task
8	document contents	shared volume -> model process	open()/read in the kitchen (service) or the Python worker (cluster); OCR via a tesseract subprocess	32.4 KB for all 6 files	each document
9	results	Python worker -> executor -> driver JVM -> driver Python -> JSON file	pickle frame; DirectTaskResult in a StatusUpdate RPC; local socket (collectAndServe); json.dump	task results 16,859 B + 18,433 B; file 57.2 KB	each task / job
10	control	driver, master, worker, executor	RPC: RegisterApplication, LaunchExecutor, RegisterExecutor, heartbeats; event log file	event log 197.8 KB for both jobs	continuous

Network bytes between the containers during the service-mode job (eth0 counters over the probes' 151-s window, which covers all but the last 16 s of the job): master node received 52.8 KB / sent 46.1 KB; worker node received 65.9 KB / sent 54.5 KB. The models did not move at all - they were already in the node caches (path 1 happened on an earlier run) and on the GPU (path 2 at model-server start).

2. Where the data comes from

2.1 The documents

The six test documents live in the repository (`data/ner_samples/`, tracked in git). Docker mounts the repository's data folder into every container at `/app/data`; on a real cluster this is a shared file system (NFS or similar) mounted at the same path on every node. Only the path string travels through Spark; the process that runs the models opens the file itself.

file	bytes	how text is obtained	time	characters	language	route	chunks
sample_scan6.png	21,390	tesseract OCR (900x300 RGB)	2.901 s	161	en	direct	1
sample_text1.txt	1,393	plain text read	0.006 s	1,381	en	direct	1
sample_text2.txt	1,505	plain text read	0.003 s	1,501	en	direct	2
sample_text3.txt	114	plain text read	0.004 s	114	en	direct	1
sample_text4.txt	3,535	plain text read	0.003 s	1,421	hi	translate (hin_Deva)	1
sample_text5.txt	4,485	plain text read	0.004 s	1,790	mr	translate (mar_Deva)	2

OCR languages actually used (packs installed in the image): eng+mar+hin+tel+tam+kan+ben+guj+pan+mal+urd. Language id: py3langid on the first 1,000 characters; en/fr/de/es/it/pt go straight to GLiNER, languages with an NLLB code are translated to English first.

2.2 The models

model	on disk (node cache)	files over 1 MB	params	in GPU memory	used for
gliner-multi (urchade/gliner_multi-v2.1)	2.3 GB	pytorch_model.bin 1.2 GB, model.safetensors 1.2 GB	289 M	577.9 MB fp16	entity extraction (all documents)
nllb-200-distilled-600M (facebook)	2.5 GB	pytorch_model.bin 2.5 GB, tokenizer.json 17.3 MB, sentencepiece.bpe.model 4.9 MB	615 M	2.5 GB fp32	translation to English (non-Latin-script documents)
hf_cache (microsoft/mdeberta-v3-base)	4 MB	spm.model	-	-	GLiNER backbone config + tokenizer

gliner-multi holds the same weights twice: GLiNER 0.2.13 loads model.safetensors and falls back to pytorch_model.bin only when it is missing, so the .bin (1.2 GB) is copied into HDFS and into every node cache but never read - it can be dropped from the model store.

2.3 Settings

Labels (19), threshold and batch sizes are constants in `mt_ner_all_formats.py`, so they reach the model process as code, not as data: MAX_CHARS 1500, GLiNER batch 16, NLLB batch 8 / max 400 tokens, score threshold 0.35, GLiNER in fp16 on the GPU: True. In service mode the executor also broadcasts the service URL, the label override (None = defaults) and the HTTP timeout - the three small broadcasts of path 5.

3. Driver side: from the command line to a Spark job

3.1 What submit_pipeline_job.py does before Spark exists

1. Reads models/pipelines/manifest.json -> module models.pipelines.ner_translate.pipeline, service_url http://ner-translate-server:8000.
2. _collect_files('data/ner_samples'): recursive glob, sorted, files only -> 6 names; then os.path.abspath -> /app/data/ner_samples/... (executors resolve paths from their own working directory, so relative names would break in cluster mode).
3. Master: --master > SPARK_MASTER_URL > manifest master_url (if it resolves) > local[4]. Execution mode: --execution-mode service|cluster|auto.
4. create_cluster_session() builds the SparkSession (settings below), then sets the log level to ERROR - which is why the scheduler's own INFO lines are missing from the driver log and this document reads them from the event log instead.

setting	value in this run	what it does here
spark.master	spark://ner-translate-master:7077	
spark.app.name	pipeline-ner_translate	
spark.submit.deployMode	client	driver runs inside the submitting process
spark.driver.memory	1g	
spark.executor.memory	512m	
spark.executor.cores	4	cores the executor offers
spark.task.cpus	2	cores reserved per task -> 4 / 2 = 2 task slots per executor
spark.python.worker.reuse	true	keep the forked Python worker for the next task
spark.python.worker.memory	4g	
spark.rpc.message.maxSize	512	MB; results above it would go through the BlockManager
spark.driver.maxResultSize	4g	sum of all task results collect() may return
spark.network.timeout	600s	long, because model loads can block a task for minutes
spark.executor.heartbeatInterval	120s	executor -> driver liveness + metrics
spark.driver.host	ner-translate-master	
spark.driver.port	42033	driver RPC endpoint (random, from the log)
spark.executorEnv.PYTHONPATH	/app:/app/inference:/app/models:/app/data	where the Python worker imports inference/ and models/ from
spark.eventLog.enabled	true	

3.2 Python starts a JVM and talks to it (Py4J)

SparkSession.builder.getOrCreate() runs spark-submit pyspark-shell as a child process: a JVM (SparkSubmit) that hosts the real SparkContext. Python and that JVM talk over **Py4J**: every sc.parallelize, rdd.mapPartitions, collect in Python is a remote call over a localhost TCP socket with an auth token. The processes and sockets on the master node while the job ran:

pid	ppid	RSS	thr	socks	process (command line, shortened)
1	0	1 MB	1	0	/sbin/docker-init -- bash -c \$SPARK_HOME/sbin/start-master.sh -h ner-translate-master && tail -f \$SPARK_HOME/logs/*master*
7	1	1 MB	1	0	tail -f /opt/spark/logs/spark--org.apache.spark.deploy.master.Master-1-ner-translate-master.out
18	1	218 MB	49	5	java -Xmx256m org.apache.spark.deploy.master.Master --host ner-translate-master --port 7077 --webui-port 8080 -h ner-translate-master
321	0	25 MB	3	3	python submit_pipeline_job.py --pipeline ner_translate --input data/ner_samples --partitions 2 --execution-mode service --master spark://ner-translate-master:7077 --drive
327	321	299 MB	83	10	java -Xmx1g ... org.apache.spark.deploy.SparkSubmit ... pyspark-shell <- driver JVM

state	local	remote	owner pid(s)	what it is
LISTEN	127.0.0.1:44863	-	321	accumulator update server (Python side)
ESTABLISHED	127.0.0.1:54350	127.0.0.1:45175	321	Py4J call channel (Python <-> JVM)
ESTABLISHED	127.0.0.1:54354	127.0.0.1:45175	321	Py4J call channel (Python <-> JVM)
LISTEN	spark-master:7077	-	18	Spark master RPC
LISTEN	:::8080	-	18	master web UI
LISTEN	:::4040	-	327	driver web UI (SparkUI)
LISTEN	127.0.0.1:45175	-	327	Py4J gateway (JVM side)
LISTEN	spark-master:33015	-	327	driver BlockManager (broadcast / task-binary blocks)
LISTEN	spark-master:42033	-	327	driver RPC endpoint (sparkDriver) - executors connect here
ESTABLISHED	spark-master:33015	spark-worker:52630	327	executor fetching broadcast blocks
ESTABLISHED	spark-master:60616	spark-master:7077	327	application / worker <-> master
ESTABLISHED	spark-master:7077	spark-worker:48884	18	application / worker <-> master
ESTABLISHED	spark-master:7077	spark-master:60616	18	application / worker <-> master
ESTABLISHED	spark-master:42033	spark-worker:45426	327	executor <-> driver RPC (tasks, status updates)
ESTABLISHED	127.0.0.1:45175	127.0.0.1:54354	327	Py4J call channel (Python <-> JVM)
ESTABLISHED	127.0.0.1:45175	127.0.0.1:54350	327	Py4J call channel (Python <-> JVM)

Start-up as logged by the driver (before the log level drops to ERROR):

UTC time	driver log	meaning
09:01:27	SparkContext: Submitted application	application submitted
09:01:28	'sparkDriver' on port 42033	driver RPC endpoint up
09:01:28	MemoryStore started with capacity 434.4 MiB	driver MemoryStore
09:01:29	Start Jetty 0.0.0.0:4040 for SparkUI	Spark UI :4040
09:01:29	Successfully created connection to ner-translate-master/172.21.0.2:7077 after	TCP connection to the master
09:01:30	Connected to Spark cluster with app ID app-20260927090130-0000	master accepted the application
09:01:30	Executor added: app-20260927090130-0000/0 on worker-20260927085758-172.21.0.4-	master placed an executor on the worker
09:01:30	Granted executor ID app-20260927090130-0000/0 on hostPort 172.21.0.4:34219 wit	executor granted
09:01:31	Logging events to	event log opened
09:01:34	is now RUNNING	executor JVM running on the worker

3.3 parallelize: six names become two partitions

`sc.parallelize(paths, 2)` (text_pipeline_engine.py). PySpark picks `batchSize = max(1, min(len(c) // numSlices, 1024)) = min(6 // 2, 1024) = 3`, pickles the list in batches of 3 into a temp file in the driver's local dir and asks the JVM (`PythonRDD.readRDDFromFile`) to load it as byte arrays - an RDD of pickled batches, sliced into 2 partitions. The paths are sorted, so:

partition	task	paths (in this order)	went to
0	TID 0	sample_scan6.png, sample_text1.txt, sample_text2.txt	one POST /predict (kitchen log: scan6, text1, text2 together)
1	TID 1	sample_text3.txt, sample_text4.txt, sample_text5.txt	one POST /predict (text3, text4 hi, text5 mr)

The pickled partition data is part of the `ParallelCollectionRDD` partition object, so it travels **inside the serialized task** (path 5), not through a separate broadcast.

3.4 mapPartitions + collect: the job

`paths_rdd.mapPartitions(process_partition)` only builds a `PipelinedRDD` in Python; nothing runs. `.collect()` makes Python cloudpickle `process_partition` (with everything it closes over: `load_fn/run_fn` or the broadcast handles) into a command, wrap it in a JVM `PythonFunction` and call `PythonRDD.collectAndServe(rdd)` over Py4J. In the JVM that is an ordinary `rdd.collect()`:

RDD id	class	created by (call site)	parents	partitions
1	PythonRDD	collect at /app/inference/text_pipeline_engine.py:167	[0]	2
0	ParallelCollectionRDD	readRDDFromFile at PythonRDD.scala:289	[]	2

The DAGScheduler cuts the lineage into stages at shuffles; there is none, so **1 job -> 1 ResultStage ("collect at /app/inference/text_pipeline_engine.py:167") -> 2 tasks**, one per partition. The stage's call stack in the event log shows the Python -> Py4J -> JVM hop:

```
org.apache.spark.rdd.RDD.collect(RDD.scala:1048)
org.apache.spark.api.python.PythonRDD$.collectAndServe(PythonRDD.scala:195)
org.apache.spark.api.python.PythonRDD.collectAndServe(PythonRDD.scala)
java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
java.base/java.lang.reflect.Method.invoke(Method.java:569)
py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)
py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:374)
```

The DAGScheduler serialises (RDD chain + function) once into the **task binary** and broadcasts it (broadcast 3, 5.2 KiB as bytes). The TaskScheduler then offers the tasks to the executor's free slots: 4 cores / spark.task.cpus 2 = 2 slots, so both tasks launch at once (locality PROCESS_LOCAL - the data is in the task itself).

4. Worker side: executor JVM and Python worker

4.1 The executor is a separate JVM, forked by the worker daemon

When the master grants an executor, the worker daemon starts `CoarseGrainedExecutorBackend` as a child JVM. Its command line (from the executor log) - note the driver URL it must call back and the 512 MB heap of service mode:

```
/usr/lib/jvm/java-17-openjdk-amd64/bin/java
-cp /opt/spark/conf:/opt/spark/jars/*
-Xmx512M
-Dspark.network.timeout=600s
-Dspark.driver.port=42033
-Dspark.rpc.message.maxSize=512
-Djava.net.preferIPv6Addresses=false
-XX:+IgnoreUnrecognizedVMOptions
-Djdk.reflect.useDirectMethodHandle=false
org.apache.spark.executor.CoarseGrainedExecutorBackend
--driver-url spark://CoarseGrainedScheduler@ner-translate-master:42033
--executor-id 0
--hostname 172.21.0.4
--cores 4
--app-id app-20260927090130-0000
--worker-url spark://Worker@172.21.0.4:34219
--resourceProfileId 0
(15 --add-opens flags for Java 17 omitted)
```

UTC time	executor log	meaning
09:01:40	CoarseGrainedExecutorBackend: Started daemon with process name: 116@ner-translate-worker	JVM up (pid@host)
09:01:42	TransportClientFactory: Successfully created connection to ner-translate-master/172.21.0.2:42033 after 144 ms	TCP to the driver / worker
09:01:43	TransportClientFactory: Successfully created connection to ner-translate-master/172.21.0.2:42033 after 9 ms	TCP to the driver / worker
09:01:45	TransportClientFactory: Successfully created connection to /172.21.0.4:34219 after 6 ms	TCP to the driver / worker
09:01:45	CoarseGrainedExecutorBackend: Successfully registered with driver	driver accepted the executor
09:01:45	NettyBlockTransferService: Server created on 172.21.0.4:43917	executor BlockManager port
09:01:45	CoarseGrainedExecutorBackend: Got assigned task 0	LaunchTask received
09:01:45	CoarseGrainedExecutorBackend: Got assigned task 1	LaunchTask received
09:01:45	Executor: Running task 0.0 in stage 0.0 (TID 0)	TaskRunner thread starts the task
09:01:45	Executor: Running task 1.0 in stage 0.0 (TID 1)	TaskRunner thread starts the task
09:01:45	TorrentBroadcast: Started reading broadcast variable 3 with 1 pieces	fetch broadcast from the driver
09:01:45	TransportClientFactory: Successfully created connection to ner-translate-master/172.21.0.2:33015 after 6 ms	TCP to the driver / worker
09:01:46	MemoryStore: Block broadcast_3_piece0 stored as bytes in memory (estimated size 5.2 KiB)	broadcast piece received
09:01:46	TorrentBroadcast: Reading broadcast variable 3 took 430 ms	broadcast read time
09:01:50	TorrentBroadcast: Started reading broadcast variable 0 with 1 pieces	fetch broadcast from the driver
09:01:50	MemoryStore: Block broadcast_0_piece0 stored as bytes in memory (estimated size 238.0 B)	broadcast piece received
09:01:50	TorrentBroadcast: Reading broadcast variable 0 took 45 ms	broadcast read time
09:01:50	TorrentBroadcast: Started reading broadcast variable 2 with 1 pieces	fetch broadcast from the driver
09:01:50	MemoryStore: Block broadcast_2_piece0 stored as bytes in memory (estimated size 215.0 B)	broadcast piece received
09:01:50	TorrentBroadcast: Reading broadcast variable 2 took 20 ms	broadcast read time
09:01:50	TorrentBroadcast: Started reading broadcast variable 1 with 1 pieces	fetch broadcast from the driver
09:01:50	MemoryStore: Block broadcast_1_piece0 stored as bytes in memory (estimated size 201.0 B)	broadcast piece received
09:01:50	TorrentBroadcast: Reading broadcast variable 1 took 14 ms	broadcast read time
09:03:32	PythonRunner: Times: total = 105406, boot = 3562, init = 1933, finish = 99911	Python worker times (ms)
09:03:36	Executor: Finished task 0.0 in stage 0.0 (TID 0). 16859 bytes result sent to driver	result sent to the driver
09:03:59	PythonRunner: Times: total = 132668, boot = 3528, init = 1973, finish = 127167	Python worker times (ms)

UTC time	executor log	meaning
09:03:59	Executor: Finished task 1.0 in stage 0.0 (TID 1). 18433 bytes result sent to driver	result sent to the driver
09:04:00	CoarseGrainedExecutorBackend: Driver commanded a shutdown	spark.stop() on the driver

Broadcast 3 is the task binary (5.2 KiB serialized, read in 430 ms - the first fetch also opens the TCP connection to the driver's BlockManager). The three ~200-byte broadcasts are the service URL, labels and timeout; each Python worker receives their values through the local socket.

4.2 The Python worker: pyspark.daemon forks one per task

A JVM cannot run Python code, so for a PythonRDD partition the executor's PythonRunner asks PythonWorkerFactory for a worker. The first request starts `python -m pyspark.daemon`, which listens on a localhost port; for every task (the first included) the executor connects to it and the daemon **fork()**s a child that inherits the accepted socket - no new interpreter start-up, and pyspark's own modules are already imported. With `spark.python.worker.reuse=true` the child goes back to the daemon after the task instead of exiting. Worker node, mid-job:

pid	ppid	RSS	thr	socks	process (command line, shortened)
1	0	1 MB	1	0	/sbin/docker-init -- bash -c \$SPARK_HOME/sbin/start-worker.sh spark://ner-translate-master:7077 -c 4 -m 2g && tail -f \$SPARK_HOME/logs/*worker*
8	1	1 MB	1	0	tail -f /opt/spark/logs/spark--org.apache.spark.deploy.worker.Worker-1-ner-translate-worker.out
17	1	203 MB	53	5	java -Xmx256m org.apache.spark.deploy.worker.Worker --webui-port 8081 spark://ner-translate-master:7077 -c 4 -m 2g
116	17	214 MB	51	7	java -Xmx512M ... org.apache.spark.executor.CoarseGrainedExecutorBackend <- executor JVM
182	116	35 MB	1	1	python -m pyspark.daemon
185	182	48 MB	1	4	python -m pyspark.daemon <- forked Python worker (runs a task)
187	182	48 MB	1	4	python -m pyspark.daemon <- forked Python worker (runs a task)

state	local	remote	owner pid(s)	what it is
LISTEN	127.0.0.1:38919	-	182	pyspark.daemon (executor connects here)
ESTABLISHED	127.0.0.1:38919	127.0.0.1:58142	187	executor JVM <-> Python worker (path 6)
ESTABLISHED	127.0.0.1:38919	127.0.0.1:58128	185	executor JVM <-> Python worker (path 6)
ESTABLISHED	spark-worker:42826	ner-translate-server:8000	185	Python worker -> kitchen HTTP (path 7)
ESTABLISHED	spark-worker:42840	ner-translate-server:8000	187	Python worker -> kitchen HTTP (path 7)
LISTEN	spark-worker:34219	-	17	worker daemon RPC
LISTEN	:::8081	-	17	worker web UI
LISTEN	spark-worker:43917	-	116	executor BlockManager
ESTABLISHED	spark-worker:36636	spark-worker:34219	116	executor -> worker daemon (WorkerWatcher)
ESTABLISHED	spark-worker:45426	spark-master:42033	116	executor -> driver RPC (path 5 / 9)
ESTABLISHED	spark-worker:52630	spark-master:33015	116	executor -> driver BlockManager (broadcast fetch)
ESTABLISHED	spark-worker:34219	spark-worker:36636	17	executor -> worker daemon (WorkerWatcher)
ESTABLISHED	127.0.0.1:58128	127.0.0.1:38919	116	executor JVM <-> Python worker (path 6)
ESTABLISHED	127.0.0.1:58142	127.0.0.1:38919	116	executor JVM <-> Python worker (path 6)
ESTABLISHED	spark-worker:48884	spark-master:7077	17	worker daemon <-> master

4.3 What crosses the executor <-> Python worker socket (path 6)

Both directions are a byte stream of 4-byte big-endian integers, lengths and pickled payloads (Spark 3.5

PythonRunner / pyspark/worker.py):

direction	in this order	here
JVM -> Python	auth secret; partition index; Python version; barrier flag; TaskContext (stage, partition, attempt, task id, cpus, resources, local properties)	partition 0 or 1, stage 0
	SparkFiles dir; Python includes; broadcast variables (count, ids and values / file paths)	3 settings broadcasts in service mode
	the command: pickled (function, profiler, deserializer, serializer) - the cloudpickled process_partition	a few KB
Python -> JVM	data frames: [length][pickled batch] ... then END_OF_DATA_SECTION (-1)	one frame: a list of 3 absolute paths
	result frames: [length][pickled object] for every object the iterator yields	one dict {hostname, pid, times, results}
	TIMING_DATA (-3) + boot / init / finish times; spill counters; END_OF_DATA_SECTION; accumulator updates; END_OF_STREAM (-4)	logged by the executor as 'Times: total, boot, init, finish'

Measured (ms): task 0: total 105,406, boot 3,562 (daemon start + fork), init 1,933 (read header, imports, unpickle command), finish 99,911 (running process_partition = waiting for the kitchen); task 1: total 132,668, boot 3,528 (daemon start + fork), init 1,973 (read header, imports, unpickle command), finish 127,167 (running process_partition = waiting for the kitchen).

4.4 Results back to the driver (path 9)

task	launched (UTC)	deserialize	run	CPU of run	result	GC	result ser.	end
TID 0 (part. 0)	09:01:45.414	1,110 ms	106.3 s	749 ms	16,859 B	17 ms	104 ms	Success
TID 1 (part. 1)	09:01:45.490	1,110 ms	133.3 s	149 ms	18,433 B	17 ms	1 ms	Success

Run time is wall time of the task; CPU time is what the executor thread actually computed - under a second, because in service mode the task only waits for the HTTP answer. The Python worker's pickled dict comes back over the local socket; the executor serialises it into a **DirectTaskResult** (it is far below spark.task.maxDirectResultSize = 1 MB, so it rides inside the StatusUpdate RPC instead of being stored in the BlockManager) and sends StatusUpdate(FINISHED). The driver's TaskResultGetter deserialises it, the DAGScheduler hands it to the job's result handler in partition order, and when both tasks are in, collectAndServe opens a localhost socket from which Python reads the pickled list of 2 dicts. The driver merges them by file name and writes results/ner_translate_20260927_090401.json (57.2 KB).

5. Inside the model process: NER step by step on the GPU

The same code runs in the kitchen (service mode) or in the Python worker (cluster mode): `pipeline.load()` once, then `pipeline.run(loader, paths) -> process_paths_batched`. Measured with `docs/ner_internals/stage_profile.py` in the model-server image, alone on the machine, all 6 documents in one call: timers wrap the pipeline's own functions (GPU synchronised before every reading) and forward hooks record the tensors that reach the GPU.

27. Inside the model process - from file bytes to entities

Top row runs on CPU cores, bottom row on the GPU; every crossing is a PCIe copy (the kitchen, or the Python worker in cluster mode)

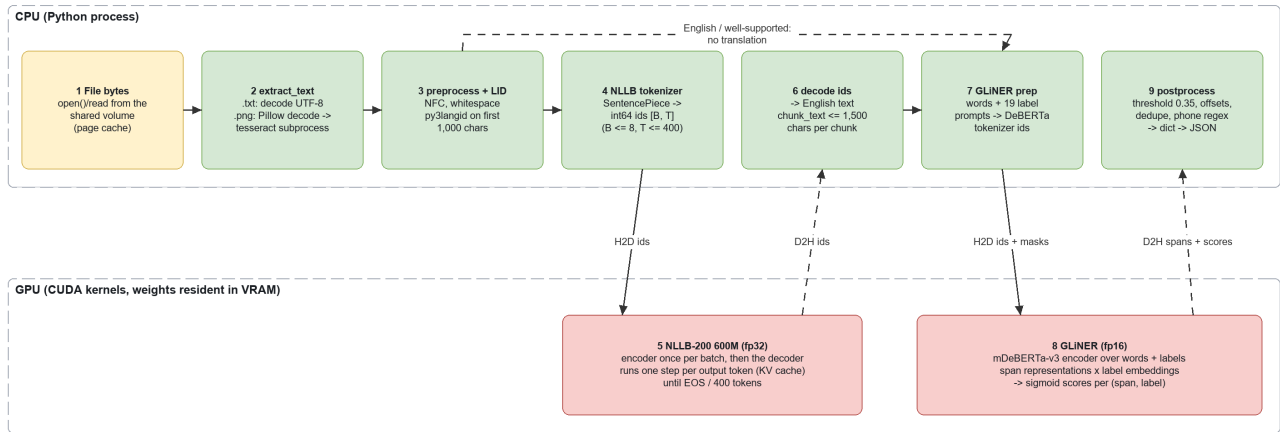


Figure 2 - from file bytes to entities; every crossing between the rows is a PCIe copy

item	value
software	torch 2.6.0+cu126, CUDA 12.6, cuDNN 90501, 8 CPU threads visible
GPU	NVIDIA GeForce GTX 1650 (Turing): 16 SMs, compute capability 7.5, 4096 MB; PCIe Gen3 x16
host -> GPU copy, 256 MB from pageable memory	4.19 GB/s (the driver stages through its own pinned buffer)
host -> GPU copy, 256 MB from pinned memory	9.03 GB/s (DMA straight from locked pages)
GPU -> host copy, 256 MB into new pageable memory	0.29 GB/s - very slow under WSL2's paravirtualised GPU; irrelevant here because only small result tensors come back

5.1 Loading the models (path 2)

step	what happens	bytes	time	1st run	VM paging during the step
resolve()	model store: .complete marker found in /model_cache -> cache hit, no HDFS call; hf_cache copied into the HF hub cache	-	0.36 s	0.20 s	-
GLINER.from_pretrained	load the tokenizer from the model folder (SentencePiece -> fast conversion: the convert_slow_tokenizer warning); build SpanModel with an mDeBERTa-v3 encoder from config (hf_cache), randomly initialised; read model.safetensors tensor by tensor (safe_open) and load_state_dict	1.2 GB	195.3 s	143.8 s	1,629 MB paged in, 266 major faults, swap in/out 0/13 MB
.to('cuda')	copy of every parameter / buffer (fp32) from pageable memory	1.2 GB	54.7 s	28.3 s	57 MB paged in, 59 major faults, swap in/out 0/0 MB
.half()	fp32 -> fp16 on the GPU (a cast kernel per tensor)	-> 577.9 MB	0.51 s	0.29 s	-
NLLB tokenizer	tokenizer.json + sentencepiece.bpe.model	22 MB	36.8 s	5.2 s	-
NLLB from_pretrained	torch.load(pytorch_model.bin, mmap=True): tensors are mapped, pages are read on first touch	2.5 GB	12.1 s	0.8 s	41 MB paged in, 21 major faults, swap in/out 0/0 MB
.to('cuda')	copy of every parameter (fp32) - touches, and so reads, the mapped pages	2.5 GB	43.7 s	97.0 s	2,828 MB paged in, 131 major faults, swap in/out 0/11 MB

Reading the table: at the measured 4.19 GB/s block-copy rate the two copies would take 0.3 s and 0.6 s. NLLB's copy read 2.8 GB from disk while it ran - the memory-mapped checkpoint is only read when the copy touches it, so this step is really the disk read (~57 MB/s from the Docker volume). GLINER's copy read only 57 MB, yet still took 54.7 s: the time is in the copy path itself - hundreds of separate pageable-memory copies through WSL2's paravirtualised GPU driver, each staged by the driver - which a single large block copy does

not show. The two profiler runs differ by up to 2x on the same steps: model loading on this laptop is dominated by the machine's I/O and memory state, not by the models or PCIe. On a worker with enough RAM and a local SSD these steps take seconds; on small nodes this is the strongest argument for loading once, in a kitchen.

GPU memory after loading: GLiNER 567 MB allocated (fp16); with NLLB 2,922 MB allocated / 2,950 MB reserved by PyTorch's caching allocator, out of 4,096 MB - one copy of the models fits, two do not (hence one task per GPU in cluster mode).

5.2 Front end: text, language, route, chunks

document	extract	chars (UTF-8 bytes)	language id	route	chunks (chars)	NLLB tokens per chunk
sample_scan6.png	2.901 s	161 (161)	en in 2,461.0 ms	direct	1 (161)	-
sample_text1.txt	0.006 s	1,381 (1,383)	en in 2.0 ms	direct	1 (1377)	-
sample_text2.txt	0.003 s	1,501 (1,503)	en in 1.0 ms	direct	2 (1338, 162)	-
sample_text3.txt	0.004 s	114 (114)	en in 0.5 ms	direct	1 (114)	-
sample_text4.txt	0.003 s	1,421 (3,533)	hi in 1.4 ms	translate hin_Deva	1 (1421)	431 (> 400: cut)
sample_text5.txt	0.004 s	1,790 (4,482)	mr in 1.0 ms	translate mar_Deva	2 (1388, 401)	444 (> 400: cut), 112

The first language-id call also loads py3langid's model (the slow first row); later calls take milliseconds. Devanagari text needs about 2.5 UTF-8 bytes per character and many more SentencePiece tokens per character than English, which matters for the next step.

Finding - translation input is truncated. `translate_chunks` tokenizes with `truncation=True`, `max_length=400`, but chunks are cut at 1,500 **characters**, not tokens. `sample_text4.txt`: chunk token counts [431]; `sample_text5.txt`: chunk token counts [444, 112]. Every token beyond 400 is dropped before NLLB sees it, so the end of that chunk is never translated and its entities are only reachable by the phone-number regex on the original text.

Fixed after this trace (`chunk_for_translation` in `mt_ner_all_formats.py`): sentences also end at the Devanagari danda / double danda and the Urdu full stop, and NLLB gets **one unit per sentence or line**, split further only above 398 tokens. Sending whole paragraphs was not enough: a token-bounded multi-sentence chunk fixed the truncation but NLLB still stopped early and dropped the sentences after the first ones. Verified on the GPU with `docs/ner_internals/verify_translate_chunking.py` (`translate_fix_20260927_151917`), same process, models loaded once, warm:

document	tokens to NLLB, before	after	unique entities before -> after
sample_text4.txt	[431]	[81, 62, 53, 45, 23, 47, 48, 33, 55]	14 -> 29; translation 725 -> 1,583 characters
sample_text5.txt	[444, 112]	[84, 61, 64, 52, 38, 53, 52, 54, 55, 27, 13, 23]	24 -> 39; translation 1,415 -> 1,892 characters
all 6 documents	translate 14.81 s, total 23.33 s	translate 9.7 s, total 15.67 s	English documents: 8 -> 8, 28 -> 27, 33 -> 33, 6 -> 6

Before the fix NLLB returned only part of each Devanagari document (the old Hindi translation covers only its first few sentences); after it, every sentence is translated - aircraft, call signs, people and places that were missing are now found. Shorter units also make translation faster: the decoder runs as many steps as the longest output in a batch, and a sentence's output is short.

5.3 Cold (first call): 59.01 s for all 6 documents

Time inside each stage function: `extract_text` 1.66 s, `detect_language` 0.00 s, `translate_chunks` 27.02 s, `predict_chunks` 30.31 s. NLLB: 2 encoder passes, 438 decoder steps. Peak GPU memory above the weights (activations, KV cache, workspaces): 284.6 MB. Result dict as JSON: 40.7 KB.

call (in order)	input on the GPU	output	time
NLLB generate hin_Deva -> eng_Latn	input_ids [1, 400] int64 (400 real tokens)	output_ids [1, 178] (steps = longest output)	17.93 s
NLLB generate mar_Deva -> eng_Latn	input_ids [2, 400] int64 (512 real tokens)	output_ids [2, 262] (steps = longest output)	8.98 s
GLiNER batch_predict_entities	7 chunks pooled from all documents: 161, 1377, 1338, 162, 114, 725, 1415 chars	entities per chunk [8, 28, 32, 3, 6, 14, 27]	30.31 s

5.4 Warm (second call): 21.24 s for all 6 documents

Time inside each stage function: `extract_text` 2.49 s, `detect_language` 0.12 s, `translate_chunks` 15.80 s, `predict_chunks` 2.82 s. NLLB: 2 encoder passes, 438 decoder steps. Peak GPU memory above the weights (activations, KV cache, workspaces): 276.5 MB. Result dict as JSON: 40.7 KB.

call (in order)	input on the GPU	output	time
NLLB generate hin_Deva -> eng_Latn	input_ids [1, 400] int64 (400 real tokens)	output_ids [1, 178] (steps = longest output)	6.49 s
NLLB generate mar_Deva -> eng_Latn	input_ids [2, 400] int64 (512 real tokens)	output_ids [2, 262] (steps = longest output)	9.19 s
GLiNER batch_predict_entities	7 chunks pooled from all documents: 161, 1377, 1338, 162, 114, 725, 1415 chars	entities per chunk [8, 28, 32, 3, 6, 14, 27]	2.82 s

What GLiNER's forward pass receives (hook on `SpanModel1.forward`): `input_ids / attention_mask / words_mask [7, 456] int64` - 7 chunks padded to 456 sub-word tokens (the 19 label prompts are prepended to every chunk's words); `span_idx [7, 3276, 2]` - every span of 1 to 12 words starting at every word, $3,276 = 273 \text{ words} \times 12 \text{ widths}$ for the longest chunk; `span_mask [7, 3276] bool`. The model scores all 3,276 spans against all 19 labels (62,244 sigmoid scores per chunk) in one fp16 pass; spans above 0.35 survive, overlaps are resolved greedily, and only those (start, end, label, score) tuples are copied back to the host.

Cold vs warm: GLiNER 30.3 s -> 2.8 s, NLLB 27.0 s -> 15.8 s. The first call pays for loading CUDA kernels (lazy module loading), creating cuBLAS / cuDNN handles and growing the caching allocator. NLLB stays expensive when warm because generation is sequential: one decoder pass per output token.

Unique entities per document: `sample_scan6.png` 8, `sample_text1.txt` 28, `sample_text2.txt` 33, `sample_text3.txt` 6, `sample_text4.txt` 14, `sample_text5.txt` 24.

5.5 What the kitchen did during the Spark job

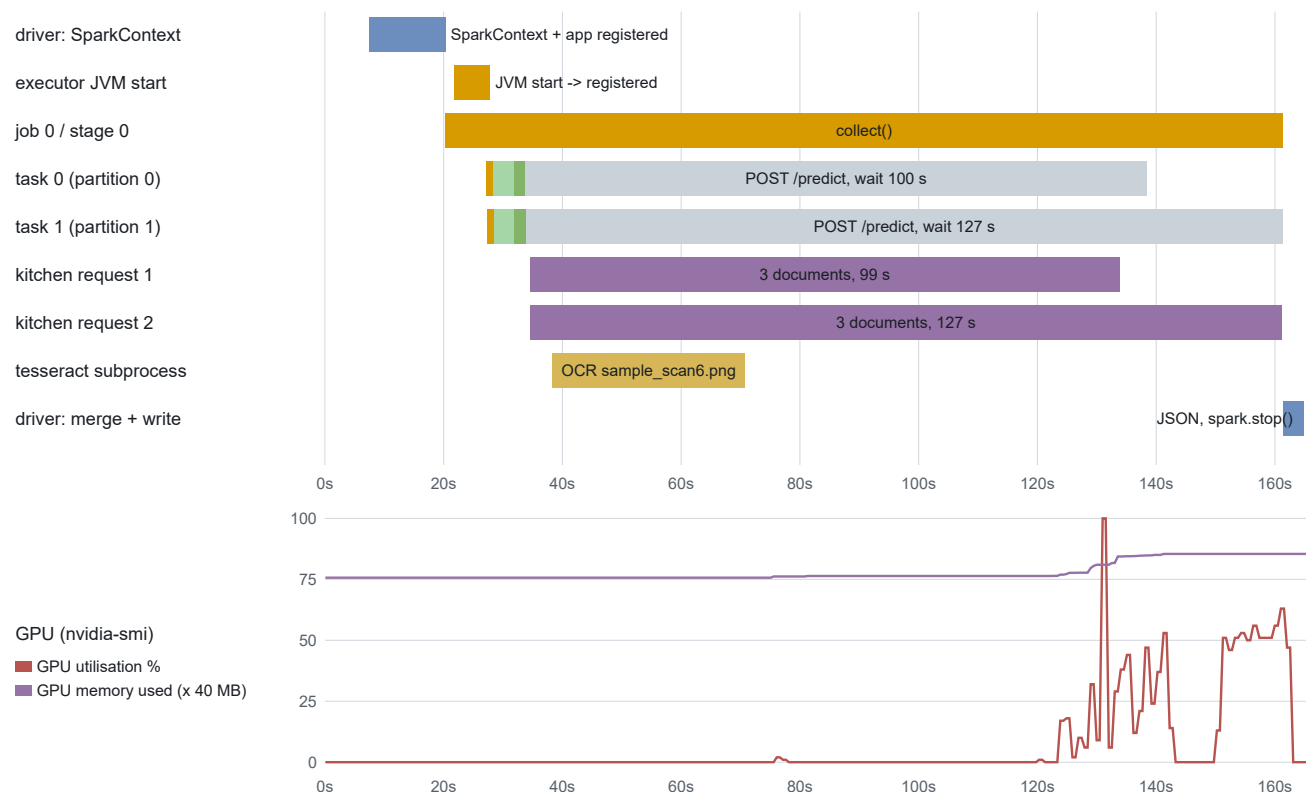
UTC time	t (s)	kitchen log
09:02:31.920	73.7	Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum len
09:03:31.970	133.8	[done] /app/data/ner_samples/sample_scan6.png lang=en route=direct -> 8 unique entities
09:03:31.971	133.8	[done] /app/data/ner_samples/sample_text1.txt lang=en route=direct -> 27 unique entities
09:03:31.973	133.8	[done] /app/data/ner_samples/sample_text2.txt lang=en route=direct -> 33 unique entities
09:03:31.977	133.8	INFO:ner_translate.serve:Processed 3 path(s) in 99.20s
09:03:32.016	133.8	INFO: 172.21.0.4:42840 - "POST /predict HTTP/1.1" 200 OK
09:03:59.295	161.1	[done] /app/data/ner_samples/sample_text3.txt lang=en route=direct -> 6 unique entities
09:03:59.296	161.1	[done] /app/data/ner_samples/sample_text4.txt lang=hi route=translated -> 14 unique entities
09:03:59.297	161.1	[done] /app/data/ner_samples/sample_text5.txt lang=mr route=translated -> 24 unique entities
09:03:59.298	161.1	INFO:ner_translate.serve:Processed 3 path(s) in 126.51s
09:03:59.301	161.1	INFO: 172.21.0.4:42826 - "POST /predict HTTP/1.1" 200 OK

The probe saw a tesseract child of the kitchen from `t = 38 s` to `70 s`: pytesseract writes the decoded image to `/tmp/tess_*.PNG`, runs the tesseract binary with all 11 language models (eng+mar+hin+...), and reads the text back - one more copy of the document inside the node.

Why the two Spark requests took 99 s and 127 s while the same batches sent directly afterwards took 4.3 s and 19.4 s: they were the **first** requests after the model server started (first OCR run loads 11 tesseract language models; first CUDA calls load kernels and create cuBLAS / cuDNN handles; the allocator grows) and they arrived **together**: FastAPI runs a plain def endpoint in a thread pool, so both requests ran at the same time in two threads, sharing one copy of the models, one GIL, 8 CPU threads and one GPU.

6. Service mode end to end - measured timeline

t = 0 is `docker exec ... submit_pipeline_job.py` (09:01:18 UTC). Bars from the event log, executor log, kitchen log and probes:



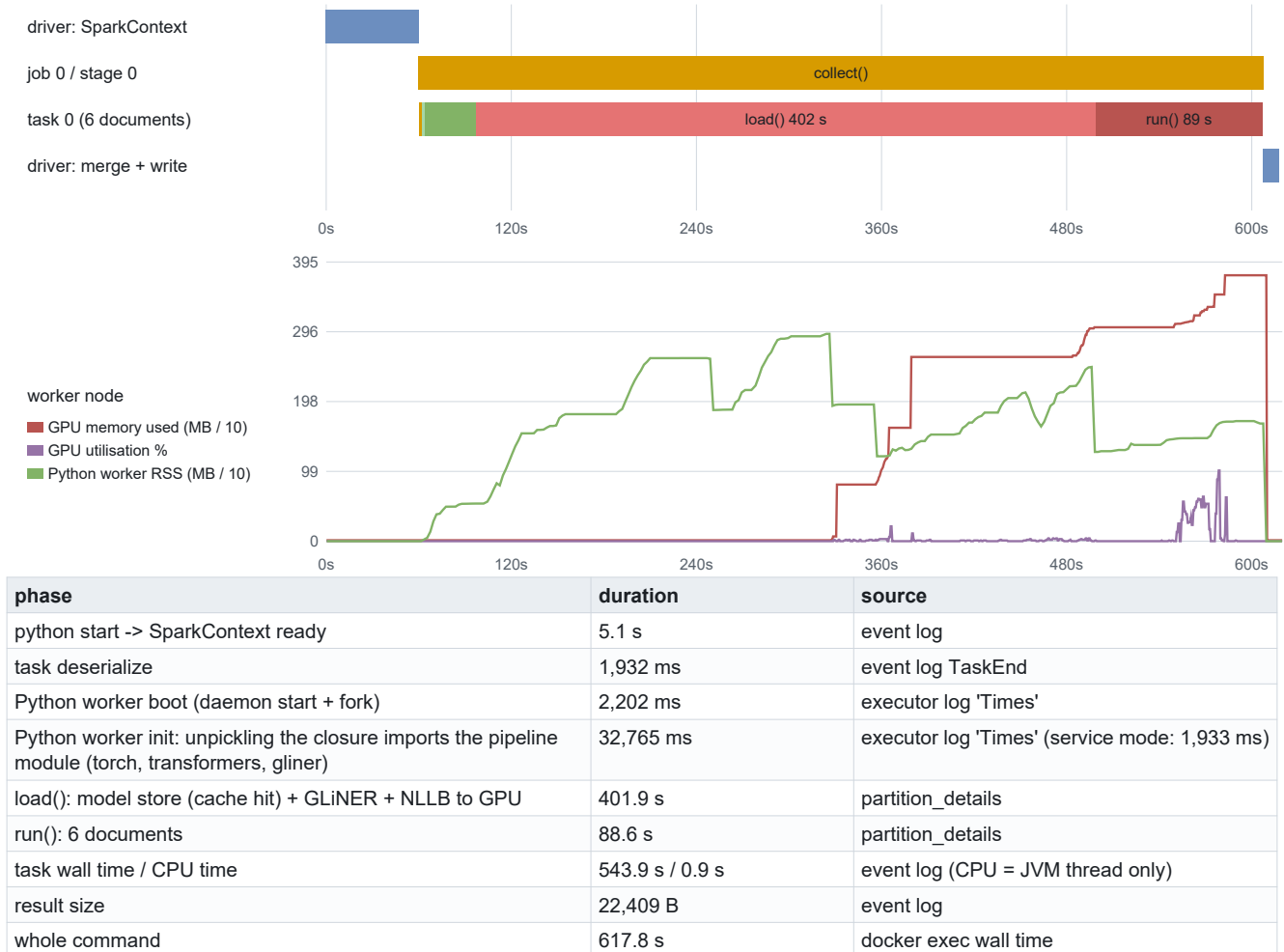
GPU memory stayed at 3,027-3,417 MB (models resident since kitchen start); utilisation peaked at 100 %; link Gen3 x16.

The GPU was idle (< 5 %) until t = 124 s although both requests reached the kitchen at t = 33 s: most of their time was CPU-side - OCR (the tesseract bar), the first-call initialisation of CUDA kernels and libraries, and two request threads taking turns on one GIL in a memory-starved VM. Once warm, the same six documents need about 21 s in total (section 5.4).

phase	duration	source
python start -> SparkContext ready	8.4 s	event log ApplicationStart
app start -> executor registered	18.7 s	event log ExecutorAdded
job submitted -> tasks launched	6.9 s	event log
task wall time (max)	133.3 s	event log TaskEnd
job (collect) total	141.0 s	event log JobStart/JobEnd
whole command	164.8 s	docker exec wall time

7. Cluster mode end to end - measured timeline

Same command with `--execution-mode cluster --partitions 1` on the cluster profile (ner-translate-worker image: Spark + torch + GLiNER + NLLB in one image). There is no kitchen: the Python worker imports the pipeline and calls `load()` then `run()` itself, so paths 2 (cache -> RAM -> GPU) and 8 (reading documents) happen **inside the Spark task**. One partition, because two concurrent tasks would each load their own copy of the models into the 4 GB GPU.



The Python worker grew to 2,934 MB RSS while loading (checkpoint tensors in RAM before the copy to the GPU). The executor JVM's own CPU time stays small: all work happens in the forked Python process, which Spark accounts for only as wall time.

Worker node processes near the end of the task:

pid	ppid	RSS	thr	socks	process (command line, shortened)
1	0	0 MB	1	0	/sbin/docker-init -- /opt/nvidia/nvidia_entrypoint.sh bash -c \$SPARK_HOME/sbin/start-worker.sh spark://ner-cluster-master:7077 -c 4 -m 6g && tail -f \$SPARK_HOME/logs/*wor
7	1	1 MB	1	0	tail -f /opt/spark/logs/spark--org.apache.spark.deploy.worker.Worker-1-ner-cluster-worker.out
36	1	185 MB	53	5	java -Xmx256m org.apache.spark.deploy.worker.Worker --webui-port 8081 spark://ner-cluster-master:7077 -c 4 -m 6g
129	36	203 MB	47	6	java -Xmx4096M ... org.apache.spark.executor.CoarseGrainedExecutorBackend <- executor JVM
204	129	46 MB	1	1	python -m pyspark.daemon
208	204	2,198 MB	7	3	python -m pyspark.daemon <- forked Python worker (runs a task)

What the Python worker printed - it lands in the executor's **stderr** file of the Spark work dir (the worker's stdout is redirected there), next to the executor's own log lines:

```
[ner_translate] model source: model store hdfs://hdfs-namenode:8020/models/weights
[model_store] cache hit /model_cache/hdfs-namenode/models/weights/gliner-multi
[model_store] cache hit /model_cache/hdfs-namenode/models/weights/nllb-200-distilled-600M
[model_store] cache hit /model_cache/hdfs-namenode/models/weights/hf_cache
[ner_translate] offline HF cache: models--microsoft--mdeberta-v3-base -> /root/.cache/huggingface/hub
GPU detected: NVIDIA GeForce GTX 1650
GLiNER using fp16.
[done] /app/data/ner_samples/sample_scan6.png lang=en route=direct -> 8 unique entities
[done] /app/data/ner_samples/sample_text1.txt lang=en route=direct -> 28 unique entities
[done] /app/data/ner_samples/sample_text2.txt lang=en route=direct -> 33 unique entities
[done] /app/data/ner_samples/sample_text3.txt lang=en route=direct -> 6 unique entities
[done] /app/data/ner_samples/sample_text4.txt lang=hi route=translated -> 14 unique entities
[done] /app/data/ner_samples/sample_text5.txt lang=mr route=translated -> 24 unique entities
```

VM-wide paging while the task ran (sampled every 2 s): 6.9 GB read from disk, swap in / out 16 / 83 MB, lowest MemAvailable 1,635 MB. The models the task needs are 3.6 GB (model.safetensors 1.2 GB + NLLB 2.5 GB), so pages were read more than once: under memory pressure the page cache evicted checkpoint pages before the copy to the GPU had used them. The load is bound by disk reads in a memory-starved VM, not by PCIe.

8. Service mode vs cluster mode

	service mode (waiter / kitchen)	cluster mode (in-process)
who reads the documents (path 8)	the kitchen	the Python worker of each task
where models live in RAM / GPU (path 2)	once, in the kitchen, loaded at server start	in every Python worker, loaded by every task
executor image	spark-lean (no torch)	ner-translate-worker (Spark + torch + models' code)
what the Python worker sends over the network	HTTP POST with 3 paths (134 B body)	nothing - it reads files and uses the local GPU
parallelism limit	kitchen threads share one model copy	GPU memory: one model copy per concurrent task
this run	142 s job, first requests after server start, 2 tasks	548 s job: load 402 s + run 89 s, 1 task
unique entities per document	scan6 8, text1 27, text2 33, text3 6, text4 14, text5 24	scan6 8, text1 28, text2 33, text3 6, text4 14, text5 24 (text1 differs by one: GPU batch composition + fp16, a mention at the 0.35 threshold)

9. Nitty-gritty findings

- **Spark moves names, not documents.** The task for 3 documents carries 3 path strings; the only large payloads Spark moves are the result dicts (16,859 B + 18,433 B). Every node must see `/app/data/...` at the same path.
- **Hindi / Marathi translations lost text - fixed** (section 5.2): chunks were sized in characters (1,500) but NLLB input is truncated at 400 tokens (the Devanagari chunks were 431-444 tokens), and multi-sentence input made NLLB stop early. Now one unit per sentence: Hindi entities 14 -> 29, Marathi 24 -> 39, translation faster.
- **gliner-multi's pytorch_model.bin is never read** (GLiNER 0.2.13 prefers model.safetensors): 1.2 GB stored in HDFS, fetched into every node cache and never used.
- **Results are keyed by file base name** (`os.path.basename`): two inputs with the same name in different folders overwrite each other in the merge.
- **Result size grows with document size:** each result carries `original_text` and `text_used` besides the entities. Past 1 MB per task a result goes through the BlockManager (IndirectTaskResult); past `spark.driver.maxResultSize` (4 GB) `collect()` fails - for large corpora write per-partition outputs instead of collecting.
- **In cluster mode load() runs once per task, not once per executor** (`text_pipeline_engine` calls `load_fn` inside `process_partition`): with worker reuse the same Python process reloads 3.6 GB of weights into the GPU for every task (402 s here). Caching the loaded models in a module-level variable of the pipeline would make it once per Python worker.
- **spark.task.cpus = 2** turns a 4-core executor into 2 task slots; in service mode both tasks' requests reach the kitchen at the same time and run concurrently in its thread pool on one model copy.
- **The executor's CPU time is not the work:** Spark's task CPU metric counts only the JVM thread; the Python worker (cluster mode) or the kitchen (service mode) does the computing.
- **First requests are slow:** tesseract language models, CUDA kernels and allocator warm-up; the stage profile's cold vs warm runs show the gap.

- **Network is not the bottleneck:** a whole service-mode job moved tens of KB between nodes; model bytes move only on the first load of a node (path 1).

10. Reproduce

```
bash docs/ner_internals/run_trace.sh           # ~25 min: both jobs, /predict, stage profile
python docs/diagrams/build_diagrams.py        # figures 25-27
python docs/ner_internals/build_ner_internals_pdf.py  # this PDF from the newest results/ner_trace_*
```

Raw evidence of this document: results/ner_trace_20260927_142727/ - eventlog/ (Spark), svc_* and clu_* (driver logs, executor work dirs, probe JSONL, GPU CSV), predict_bytes.txt, stage_profile.json.